GCP TRAINING PROGRAM — MODULE 7 HANDBOOK

GenAI Model Evaluation
A Practical Handbook

Companion guide to *genai_evaluation_framework_lab.ipynb* — the Gen AI evaluation service on the Gemini Enterprise Agent Platform (formerly Vertex AI)

PREPARED FOR	Himanshu — Trainer & Curriculum Developer, Talktotechie Services Pvt. Ltd.
PLATFORM	Google Cloud — Gemini Enterprise Agent Platform (Vertex AI)
PROJECT	himanshugcpproject

Contents

1 Why GenAI Model Evaluation Is a Discipline, Not a Vibe Check

A support team wants to auto-draft replies to customer messages using Gemini. Before this ships to production, someone needs a repeatable, quantitative way to answer three questions that come up in essentially every real GenAI deployment:

- **Is a given model/prompt actually good enough?** — scored against fixed criteria, not eyeballed by whoever happens to read the output that day.
- **Is a cheaper model good enough to replace a more expensive one?** — a *model migration* decision, backed by evidence rather than a guess.
- **Did a prompt change make things better or worse?** — a *prompt regression test*, run the same way every time a prompt is edited.

This is the same motivation as unit tests in software engineering — except the “correct output” for a language model is a matter of degree and judgment, not an exact-match pass/fail. A response can be perfectly correct while using completely different words than any reference answer. The Gen AI evaluation service on the Gemini Enterprise Agent Platform is Google's structured way to manage that ambiguity: it turns “this reply looks fine to me” into a scored, versioned, reviewable artifact that can be compared across models, prompts, and time.

> **✦ TRAINER TIP**
> Frame this for trainees the way a QA engineer would frame test coverage: an evaluation set is a test suite, metrics are assertions, and an experiment is a test run you can point back to months later when someone asks “why did we choose this model?”

The rest of this handbook walks through the companion notebook (*genai_evaluation_framework_lab.ipynb*) section by section, then closes with a glossary and a set of evaluation best practices that go beyond what any single notebook can show.

2 Notebook Walkthrough — Section by Section

2.1 Setup: Authenticate, Install, Configure

The notebook opens with Colab authentication (or a reminder that local `gcloud auth application-default login` credentials are assumed), then installs `google-cloud-aiplatform[evaluation]`. The `[evaluation]` extra matters — it is what pulls in the eval-specific SDK surface (`vertexai.Client().evals`, rubric metrics, etc.) rather than just the base Vertex SDK. A kernel restart immediately follows the install; this is a normal Colab pattern since freshly installed packages aren't always picked up until the runtime resets — worth flagging so trainees don't mistake it for an error.

Project configuration sets the GCP project, enables the `aiplatform.googleapis.com` API, and instantiates `vertexai.Client(project=PROJECT_ID, location=REGION)`. This `client` object is the newer-generation Gen AI SDK client used in Sections 4 and 6 below (`client.evals.run_inference`, `client.evals.evaluate`).

> ✦ **TRAINER TIP**
> The notebook actually mixes two SDK generations: the newer `vertexai.Client().evals` (Sections 4 and 6) and the older `vertexai.evaluation.EvalTask` (Sections 5, 7, 8). Both are valid and both currently ship — this is a natural, honest place to mention that the SDK is mid-transition, consistent with the wider Vertex AI → Gemini Enterprise Agent Platform rebrand announced at Google Cloud Next 2026.

2.2 The Evaluation Dataset

Eight short customer messages, each paired with a `reference` — the reply a good human agent would send. This reference is what makes both computation-based metrics (2.3, which compare wording directly) and some model-based metrics (2.4, which use an LLM judge) possible. Without ground truth to compare against, evaluation is only ever unverified opinion — the same reason a labeled test set matters in classical ML.

2.3 Step 1 — Generate, Then Score (Computation-Based Metrics)

Generate responses (`run_inference`): sends all eight prompts to `gemini-2.5-flash` and collects the responses into `eval_dataset_v1`. This is the first half of the evaluation service's deliberately two-step process: generate, then separately evaluate. Keeping the steps apart is what lets the model-comparison step (2.5) reuse the same generated responses against multiple metrics without regenerating them each time — saving both cost and API calls.

ROUGE and BLEU: pure word-overlap scoring against the reference column, no model judgment involved — fast, deterministic, and essentially free. `rouge_1` measures overlapping single words; `bleu` measures overlapping short phrases (n-grams). The limitation worth flagging hard in class: a reply can be excellent and still score low here if it's well-written but phrased completely differently from the reference — which is exactly why the next step layers model-based judgment on top.

2.4 Step 2 — Model-Based Metrics (LLM-as-Judge)

Rather than counting overlapping words, a judge model (Gemini, used internally by the evaluation service) reads the prompt, the reference, and the candidate response, then scores it against a defined rubric — much closer to how a human reviewer would actually assess quality. Two built-in rubric metrics are used here:

- **TEXT_QUALITY** — overall coherence, grammar, and clarity.

- **INSTRUCTION_FOLLOWING** — whether the response actually addresses what was asked, not just whether it reads well.

> ■ CONSOLE DEMO
> **Where to demonstrate this**
> Agent Platform (Vertex AI) console → **Models** → **Evaluation**. This run appears listed with its summary scores; clicking in shows the same per-example breakdown as `result.summary_metrics` in the notebook — without needing the notebook open. Good moment to show trainees that eval history persists outside the notebook, useful for audits later.

2.5 Comparing Two Models (a Model Migration Decision)

The real question this answers: “Can we switch from `gemini-2.5-pro` to the cheaper, faster `gemini-2.5-flash-lite` for this task without a meaningful quality drop?” Both models run against the identical prompt set, then get scored side by side on `TEXT_QUALITY` and `INSTRUCTION_FOLLOWING` using `client.evals.evaluate(dataset=[inference_pro, inference_flash_lite], ...)`.

> ✓ HOW TO READ IT
> **How to read it**
> If `gemini-2.5-flash-lite` scores within a small margin of `gemini-2.5-pro` on both metrics, that's quantitative evidence supporting a migration to the cheaper model for this specific task — turning a subjective “seems fine to me” into a defensible, reviewable decision. Pair this with the per-token price difference between the two models when presenting the business case.

2.6 Defining a Custom Metric

Built-in metrics are generic. This support team has one business-specific rule: every reply acknowledging a customer complaint should include a genuine apology or acknowledgement, not just jump straight to a solution. That's not a metric Google ships, so the notebook defines its own rubric-based `PointwiseMetric` — a custom prompt template the same judge model uses to score each response against this specific criterion, with a 1 / 3 / 5 rating rubric spelling out what each score level looks like.

The notebook also introduces a small production-hygiene helper, `generate_with_retry`, which backs off exponentially on HTTP 429 (rate limit) responses — worth calling out since trainees will hit rate limits themselves once running dozens of evaluation calls back to back.

> ✦ TRAINER TIP
> **Sanity-check for the class**
> Compare per-example scores here against the raw responses from Section 2.3. The two replies with an explicit “I'm sorry” / “I understand the frustration” opening should score higher than ones where an apology wouldn't even make sense (the restock question, the cancellation request). If everything scores the same regardless of content, the custom rubric isn't discriminating and needs rework — a good teaching moment on validating that a custom metric actually works before trusting it.

2.7 Prompt Regression Testing

The most practically reusable section. The real question: “We're about to change our production system prompt — will this make responses better or worse?” Two system prompts are compared: a bare-bones baseline (“you are a customer support agent, respond to the message”) and an improved version that explicitly instructs empathy-first phrasing, acknowledging frustration before offering a solution, and keeping replies to 2–3 sentences. Both run through `gemini-2.5-flash` and get scored on `TEXT_QUALITY` plus the custom `acknowledgement_metric` from Section 2.6.

✓ HOW TO READ IT

How to read it

`acknowledges_customer` should jump noticeably for the improved prompt; `TEXT_QUALITY` should stay roughly flat. That combination — the targeted metric moves, the general-quality metric doesn't drop — is exactly the evidence you want: the change achieved its specific goal without a general quality regression elsewhere. This cell, with any new candidate prompt swapped in, is the reusable regression test trainees should walk away with: same dataset, same metrics, one line changed.

■ CONSOLE DEMO

Where to demonstrate this

Agent Platform → **Models** → **Experiments** → **genai-eval-lab-experiment**. Every run from Sections 2.4, 2.6, and 2.7 logs into this same experiment name, so this is the one screen where baseline-vs-improved sits side by side with the earlier TEXT_QUALITY/INSTRUCTION_FOLLOWING runs — all comparable in one place. This is the best “here's your audit trail” moment in the whole lab.

2.8 Cleanup

This notebook creates no continuously-billing resources — every call (`evals`, `genai.Client`) is a pay-per-request API call with no idle cost, unlike the deployed endpoints in the companion MLOps and churn notebooks. There is nothing to delete, which is worth contrasting explicitly so trainees don't go hunting for a resource to tear down.

■ CONSOLE DEMO

Where to demonstrate this

Agent Platform → **Models** → **Evaluation** retains a history of every evaluation run as a standing record — the paper trail for why a model or prompt decision was made, tying back neatly to the unit-testing framing from Section 1.

3 Console Quick-Reference Map

One consolidated table for live demos — what to click, and what it's meant to show the class.

Agent Platform → Models → Evaluation	List of every evaluation run with summary scores; click in for per-example breakdown	2.4, 2.8
Agent Platform → Models → Experiments → genai-eval-lab-experiment	All runs sharing this experiment name side by side — model comparisons, custom metric runs, and both regression-test runs together	2.4, 2.6, 2.7
Agent Platform → Model Garden	Card view of gemini-2.5-pro / flash / flash-lite — useful for showing pricing/latency context right before the model migration comparison	2.5
IAM & Admin → APIs & Services → Enabled APIs	Confirm aiplatform.googleapis.com is enabled for the project before running the notebook live	2.1
Billing → Reports (filtered to Vertex AI / Generative AI SKU)	Actual per-call cost for the class's evaluation runs — grounds the “cheaper model” discussion in real numbers instead of an abstract claim	2.5

✦ TRAINER TIP

Since the Google Cloud Next 2026 rebrand, some of these menu labels are transitioning from “Vertex AI” to “Gemini Enterprise Agent Platform” in the console navigation. If a trainee can't find a menu item exactly where this table says, search the console top bar for “Evaluation” or “Experiments” directly — faster than hunting through a renamed left-nav mid-transition.

4 Glossary of Terms

Terms used in the notebook and in this handbook, in plain language.

Gen AI evaluation service	The Vertex AI / Agent Platform component that runs generation and scoring for GenAI outputs at scale, and stores the results as reviewable runs and experiments.
run_inference	The SDK call that sends every row of an evaluation dataset to a chosen model and collects its responses, without scoring them yet — the “generate” half of the two-step generate-then-evaluate pattern.
Computation-based metric	A metric calculated by a fixed formula comparing text directly (e.g. word overlap) — deterministic, fast, cheap, and requires no model call to compute. ROUGE and BLEU are the classic examples.
ROUGE	Recall-Oriented Understudy for Gisting Evaluation. Originally built for summarization; rouge_1 measures how many single words in the reference also appear in the candidate response.
BLEU	Bilingual Evaluation Understudy. Originally built for machine translation; measures overlap of short word sequences (n-grams) between candidate and reference, rewarding matching phrases, not just matching words.
N-gram	A contiguous sequence of N words. A 2-gram (bigram) of “I am sorry” is [“I am”, “am sorry”]. BLEU scores are built from n-gram overlap counts.
Model-based metric / LLM-as-judge	A metric computed by having another LLM (the “judge model”) read the input, and candidate response (and sometimes a reference), then output a score against a described rubric — approximating how a human reviewer would rate quality.
Rubric metric	A model-based metric defined by a scoring rubric — explicit criteria plus a description of what each score level (e.g. 1, 3, 5) looks like, so the judge model's scoring is consistent and explainable rather than an unstructured opinion.
TEXT_QUALITY	A built-in rubric metric scoring overall coherence, grammar, and clarity of a response.
INSTRUCTION_FOLLOWING	A built-in rubric metric scoring whether a response actually addresses what was asked — independent of whether it happens to read well.
PointwiseMetric	An evaluation metric that scores one response at a time against a rubric (as opposed to comparing two responses against each other). Used here to build the custom “acknowledges_customer” metric.
Pairwise metric	An evaluation metric that scores two candidate responses against each other and picks (or ranks) a winner, rather than scoring each independently. Not used in this notebook, but common where “which of these two is better” matters more than an absolute score.
PointwiseMetricPromptTemplate	The template object defining a custom pointwise metric's criteria text and rating rubric, which the judge model is instructed to follow when scoring.

EvalTask	The (older-generation) Vertex AI SDK class that bundles a dataset, a list of metrics, and an experiment name into one evaluation run via `.evaluate()`.
Experiment	A named container in the console (e.g. genai-eval-lab-experiment) that groups related evaluation runs together so they can be compared on one screen over time.
Reference / ground truth	The known-good answer an evaluation example is compared against. Enables both computation-based scoring and stronger model-based scoring; without it, evaluation is closer to unverified opinion.
Model migration	The decision to move a production workload from one model to another (typically cheaper or faster), justified by evaluation evidence that quality doesn't meaningfully drop.
Prompt regression test	Re-running the same evaluation dataset and metrics against a new prompt version to confirm it improves the intended behavior without degrading general quality — named for its parallel to regression testing in software.
Hallucination	When a model states something confident-sounding but factually incorrect or unsupported by the given context. A key risk metric in production GenAI evaluation, not directly exercised in this notebook.
Groundedness	How well a response's claims are actually supported by the provided source material (e.g. retrieved documents in a RAG system). A common model-based metric in retrieval-augmented pipelines.
Toxicity / safety metric	Scores measuring whether a response contains harmful, offensive, or unsafe content — usually run as a mandatory gate before production release, separate from quality metrics.
Judge bias	Systematic scoring distortions in an LLM-as-judge setup — e.g. favoring longer responses (verbosity bias), favoring the first option shown (position bias), or favoring its own output family (self-preference bias).
Golden dataset	A curated, versioned evaluation set treated as a stable benchmark — changed deliberately and rarely, so that score changes over time reflect model/prompt changes, not dataset drift.

5 General Best Practices for GenAI Evaluation

The notebook demonstrates one complete, well-built evaluation workflow. In production, teams layer several more practices on top of it. Worth covering with trainees as the “beyond this lab” material.

5.1 Build a Representative, Versioned Evaluation Set

- Cover the real distribution of production inputs — including edge cases, ambiguous requests, and adversarial or hostile messages — not just the easy, clean examples (this lab's 8 rows are illustrative, not comprehensive).
- Treat the evaluation set itself as a versioned artifact (a “golden dataset”). If it changes silently, score changes become impossible to attribute to the model, the prompt, or the dataset.
- Keep the evaluation set separate from anything used to write or tune the prompt — tuning a prompt against the same examples used to score it overstates quality, the GenAI equivalent of testing on the training set.

5.2 Combine Multiple Metric Types — Don't Rely on One Number

- Pair reference-based metrics (ROUGE/BLEU) with model-based judgment; each covers a blind spot in the other, as this notebook itself demonstrates.
- Add safety/policy metrics (toxicity, PII leakage, harmful content) as a hard gate, evaluated independently from quality metrics — a response should never ship on the strength of high quality scores alone if it fails a safety check.
- For retrieval-augmented (RAG) systems specifically, add groundedness / faithfulness metrics — quality metrics alone won't catch a fluent, well-structured hallucination.

5.3 Manage LLM-as-Judge Carefully

- Be aware of judge biases: verbosity bias (favoring longer answers), position bias (favoring whichever option is listed first in pairwise comparisons), and self-preference bias (a model family favoring its own outputs).
- Periodically spot-check the judge against human ratings on a sample — calculate agreement, and treat persistent disagreement as a signal to rewrite the rubric, not just to distrust the judge.
- Keep rubrics concrete and behavior-anchored (“what does a 3 look like versus a 5”) rather than abstract adjectives — exactly the pattern the custom acknowledgement metric in this notebook follows.

5.4 Design for Statistical Honesty

- Eight examples are enough to demonstrate a workflow, not enough to draw a confident production decision from — real evaluation sets typically run from dozens to thousands of examples depending on how nuanced the task is.
- Look at score distributions and variance, not just the mean — two models can have the same average score with very different consistency.
- When comparing models or prompts, ask whether the score gap is larger than the run-to-run noise you'd see from re-running the same model twice, before calling a difference meaningful.

5.5 Make Evaluation Continuous, Not a One-Time Gate

- Re-run the same evaluation suite whenever the model version, prompt, or underlying retrieval/tooling changes — the regression-test pattern in Section 2.7 should run on every prompt change, ideally wired into CI/CD rather than run manually.
- Monitor production outputs on a sample basis after launch. Offline evaluation sets can't fully anticipate real user phrasing, and model behavior can drift as providers update models behind the same name/version.
- Keep every run logged against a named experiment (as this notebook does) so decisions are traceable later — “why did we pick flash-lite over pro in March” should be answerable by opening a console screen, not by memory.

5.6 Tie Evaluation to Business Cost, Not Just Quality

- Pair every quality score with the corresponding per-token or per-call cost and latency — a model migration decision is a cost/quality tradeoff, not a quality-only decision.
- Decide acceptable quality thresholds *before* running the comparison, not after seeing the numbers — otherwise it's easy to unconsciously rationalize whichever result is cheaper.

6 Common Pitfalls Checklist

A quick pre-flight list — useful as a closing slide or a handout for trainees building their own evaluation pipelines after the lab.

- Evaluating only on easy, clean inputs and skipping ambiguous or adversarial ones.
- Using the same examples to write the prompt and to score it.
- Trusting a single metric (especially a single word-overlap metric) as the full picture.
- Never spot-checking the LLM judge against human judgment.
- Treating a small quality gap as meaningful without checking it exceeds normal run-to-run variance.
- Skipping safety/toxicity checks because quality scores looked good.
- Not versioning the evaluation dataset, so historical scores become impossible to compare fairly.
- Comparing models on quality alone, without factoring in per-call cost and latency.
- Running evaluation once at launch and never again as prompts, models, or user behavior drift.
- Not logging runs to a named experiment, losing the audit trail for why a decision was made.

Handbook prepared as a companion to genai_evaluation_framework_lab.ipynb, GCP Training Program — Module 7. Console menu labels reflect the Vertex AI / Gemini Enterprise Agent Platform naming as of mid-2026 and may continue to shift as the rebrand rolls out.